FAST NATIONAL UNIVERSITY OF COMPUTER & EMERGING SCIENCES

CS-4063: Natural Language Processing

Assignment 3 — Transformers + RAG

Total Marks: 80 + 5 Bonus
Due Date: 29-04-26, 11:59 PM

Restrictions — Read Before You Start

Both the encoder-only and decoder-only Transformer models must be implemented entirely from scratch. You are not allowed to use any pretrained models such as BERT, GPT, T5, or any library that abstracts away the internal Transformer implementation.The following built-ins are **not** allowed: `nn.Transformer`, `nn.MultiheadAttention`, `nn.TransformerEncoder`.

Transformer-based Review Understanding with RAG-Enhanced Explanation Generation

In this assignment, you will build a three-stage NLP system that first extracts structured information from text, retrieves relevant examples from the dataset, and then generates grounded natural language explanations. The three stages — encoding, retrieval, and generation — are implemented in Parts A, B, and C respectively, and must work together as a coherent pipeline.

Dataset Description

You will use the Amazon Reviews dataset, available at:

https://nijianmo.github.io/amazon/index.html

You are required to construct your own dataset subset. To ensure diversity, you must select reviews from at least three different product categories, including approximately ten to fifteen thousand reviews per category, resulting in a final dataset of between thirty thousand and forty-five thousand samples. Each sample must include the review text and its corresponding star rating on a scale of one to five.

You must divide your dataset into training, validation, and testing sets using a 70%, 15%, and 15% split respectively. You are fully responsible for preparing the dataset and ensuring it is suitable for training.

Preprocessing

All preprocessing must be implemented by you using the techniques covered in previous lectures. Your pipeline should include text cleaning where necessary, tokenization, vocabulary construction

using only the training data, conversion of tokens to numerical indices, and the application of padding and truncation with a fixed maximum sequence length of your choosing. Every step of your preprocessing pipeline must be clearly explained in your report.

Part A Encoder Model for Understanding [25 Marks]

In this part, you will implement an encoder-only Transformer model that performs multi-task learning on the Amazon Reviews dataset. The model must jointly learn to classify the sentiment of a review and predict a second derived feature that you define yourself.

The sentiment classification must use a three-class formulation: ratings 1–2 are mapped to Negative, rating 3 to Neutral, and ratings 4–5 to Positive. For the second task, you are free to define any meaningful property of a review that can be predicted from the text alone. Your choice must be clearly motivated and documented in your report.

The encoder must produce three outputs for each review: a predicted sentiment label, a predicted value for your derived feature, and a fixed-dimensional vector representation of the review. This vector representation will be used as the basis for retrieval in Part B and must be saved to disk after training.

You must train the model end-to-end with a suitable combined loss. Your training pipeline must produce learning curves and evaluation metrics for both tasks on the held-out test set. All design choices regarding architecture, training procedure, and loss formulation are your own responsibility.

Part A — Marking Rubric

Component	Marks	Key Criteria
Input representation	3	Reviews are correctly represented as numerical sequences; vocabulary built from training data only
Attention mechanism	4	Attention is correctly implemented; queries, keys and values produce meaningful alignments
Multi-head configuration	4	Multiple attention heads operate independently and their outputs are correctly combined
Encoder block design	4	Complete encoder block with normalisation, feed-forward sub-layer, and residual connections

Multi-task classification	4	Both tasks (sentiment and derived feature) are predicted from a shared encoder representation
Training pipeline	3	Model trains successfully; optimiser, schedule, and loss curves are all present and correct
Evaluation and embeddings	3	Classification metrics reported for both tasks; review embeddings saved for retrieval
Total	**25**	—

Part B Retrieval Module [15 Marks]

In this part, you will implement the retrieval layer of your RAG pipeline. The retrieval module takes the encoder's representation of a test review and identifies the most similar reviews from the training corpus, which are then used to ground the decoder's generation in Part C.

You must store the vector representations produced by your Part A encoder for the entire training set. At inference time, a query representation is constructed from the test review's encoder output and compared against all stored vectors to retrieve the top-k most similar training examples. The number of retrieved examples k should be configurable, and your choice should be justified in your report.

Your report must include an analysis of retrieval quality. Present examples of query reviews alongside their retrieved results and discuss whether the matches are semantically meaningful. You should also reflect on the limitations of your approach and how they might affect downstream generation quality. Additionally, comment on the similarity metric used (e.g., cosine similarity) and how it impacts retrieval performance. You may also explore how variations in k influence both relevance and diversity of retrieved results. Consider discussing potential improvements such as better embeddings or indexing techniques. Finally, briefly relate retrieval effectiveness to the overall performance of the RAG pipeline.

Part B — Marking Rubric

Component	Marks	Key Criteria
Embedding storage and indexing	4	All training embeddings are stored and can be accessed efficiently at inference time
Query construction	3	A meaningful query representation is built from

		the encoder output for each test review
Similarity search	4	Similar reviews are ranked and retrieved correctly based on the chosen similarity measure
Retrieval quality and integration	4	Retrieved results are relevant to the query; context is correctly passed to the decoder
Total	**15**	—

Part C Decoder Model for Explanation Generation [25 Marks]

In this part, you will implement a decoder-only Transformer model that generates a short natural language explanation for each sentiment prediction produced by the encoder. This is the generative component of your RAG pipeline.

The decoder takes as input a structured combination of four elements: the original review text, the predicted sentiment label, the predicted derived feature, and the top-k similar reviews retrieved in Part B. You must define a clear and consistent template for combining these inputs into a single sequence. The model should then generate an explanation of one to two sentences that describes why the review carries the predicted sentiment, drawing on the retrieved examples where relevant.

The decoder must be capable of generating text autoregressively, conditioning on all previously generated tokens. A mechanism must be placed to ensure the model cannot attend to future positions during training or inference. The model is trained to minimize a language modelling objective over reference explanation sequences.

For evaluation, you must report perplexity on the test set and present at least five generated examples with commentary on their quality. You must also run a comparison between the full system and a baseline that does not use retrieved context, demonstrating the contribution of the retrieval module. All architectural decisions are your own responsibility and must be justified in your report.

Hyperparameter Tuning

This assignment is first and foremost a learning exercise. You are strongly encouraged to experiment freely with different configurations to develop an intuition for how each design choice affects your model's behaviour and performance.

Best results are encouraged. Try different values for parameters such as learning rate, number of layers, hidden dimensions, number of attention heads, dropout rate, batch size, and sequence length. Document every configuration you attempt along with its effect on validation performance. Your report should reflect a genuine exploratory process and honest analysis, not merely a final result.

Part C — Marking Rubric

Component	Marks	Key Criteria
Input construction	3	All four inputs (review, sentiment, derived feature, retrieved context) are combined coherently
Decoder attention	5	Self-attention is constrained so the model cannot attend to future tokens during generation
Autoregressive generation	5	Text is generated token by token; generation terminates correctly
Decoder block design	4	Stacked decoder blocks with correct sub-layers and residual connections
Quantitative evaluation	4	Perplexity is correctly computed and reported on the held-out test set
Qualitative evaluation	2	At least five generated explanations are presented and analysed for coherence and relevance
RAG ablation study	2	A baseline (no retrieval) is compared against the full system; improvement is documented
Total	**25**	—

Overall Evaluation Rubric

Component	Marks	Notes
Part A: Encoder Model	25	See Part A rubric
Part B: Retrieval Module	15	See Part B rubric
Part C: Decoder & RAG Pipeline	25	See Part C rubric
Report Quality	10	Clarity, analysis, depth (3–5 pages)

GitHub & Incremental Commits	5	Version control discipline
TOTAL	**80**	
Bonus: UI / Visualization OR advanced features	Up to 5	Final mark capped at 80

Note: The total marks for this assignment are **80 marks**. The bonus of up to 5 marks is awarded on top of already earned marks and the final recorded mark will not exceed 80.

Deliverables

- Complete, well-commented source code submitted as a Jupyter Notebook
- Trained model weights saved in the models/ directory
- Saved encoder embeddings for the training set in the results/ directory
- Report (3–5 pages) covering all items listed below

Report Must Cover:

- Overall system design and methodology
- Justification of design decisions for each part
- Preprocessing pipeline description
- Evaluation results with tables and plots for all metrics
- Hyperparameter tuning log with analysis of results
- RAG ablation study comparing the system with and without retrieval

All code must be properly organised and runnable from top to bottom. Clear instructions for execution must be provided at the top of the notebook.

Submission Requirements

Notebook Naming Convention

`i23XXXX-NLP-Assignment2.ipynb`

Required Directory Structure

Directory	Contents
`results/`	Saved model metrics and evaluation outputs
`models/`	Trained model weights

GitHub Submission

All submissions must be made through GitHub with incremental commits. Commit history will be reviewed as part of the evaluation. Ensure that your repository reflects a logical and progressive development of the system, with meaningful commit messages at each stage.

— Good Luck —